

Assignment 1: Implement an Email Phishing Detector

Objective:

Create a script that scans email content for common phishing indicators and alerts the user to potential phishing attempts.

Requirements:

1. **Input:**
 - The script should accept a text file containing email content.
2. **Indicators:**
 - Suspicious links (e.g., URLs with uncommon domains, IP addresses in URLs).
 - Spoofed sender addresses (e.g., email addresses that look similar to legitimate ones but have minor differences).
 - Urgent language (e.g., words like "urgent," "immediately," "action required").
3. **Output:**
 - The script should print a summary indicating whether the email is likely a phishing attempt and list the detected indicators.

Bonus:

- Implementing a user interface (UI) to upload the email content and display the results.
- Implementing an add-on for Gmail that integrates the phishing detection functionality directly within the Gmail interface, allowing users to scan emails for phishing attempts directly from their inbox.

Assignment 2: Develop a Malware Analysis Sandbox

Objective:

Create a sandbox environment that executes and monitors a sample piece of malware, capturing its behavior.

Requirements:

1. **Sandbox Setup:**
 - Use a virtual machine or container to isolate the malware.
 - Monitor file system changes, network connections, and process activity.
2. **Execution and Monitoring:**
 - Execute the malware within the sandbox.

- Log its behavior, including file modifications, created/deleted files, network activity, and spawned processes.
- 3. **Report:**
 - Generate a report summarizing the malware's behavior.

Bonus: Implementing a user interface (UI) to start/stop the sandbox and view the generated report.

Assignment 3: Simulate a SQL Injection Attack

Objective:

Develop a simple web application and demonstrate a SQL injection attack, then implement measures to prevent it.

Requirements:

1. **Web Application:**
 - Create a basic web application with a login form.
 - Use a database to store user credentials.
2. **SQL Injection:**
 - Demonstrate a SQL injection attack that bypasses the login form.
3. **Mitigation:**
 - Implement measures to prevent SQL injection (e.g., parameterized queries, prepared statements).

Example Output:

1. **Vulnerable Login:**
 - Demonstrate a SQL injection attack: ' OR '1'='1
 - Successful login without valid credentials.
2. **Secure Login:**
 - Attempt the same SQL injection attack.
 - Login should fail, demonstrating the effectiveness of the mitigation.

Bonus: Implementing a user interface (UI) to demonstrate the vulnerable and secure login forms and show the attack results.